

VOLUME 2 · B.TECH CSE STUDY SERIES

C++ PROGRAMMING

Complete Professional Handwritten Notes

From Foundations to Advanced Object-Oriented Architecture, VTABLE Internals, Modern C++ Smart Pointers & Full STL Mastery

```
#include <iostream>
class Node { public:
    virtual void draw();
}; // Modern C++
```

✓ OOP Principles
(Encapsulation/Polymorphism)

✓ VTABLE & VPTR Deep Dive

✓ Complete STL Containers &
Iterators

✓ Smart Pointers
(unique/shared/weak)

✓ 30+ OOP & STL Interview Q&A

✓ Real-World OOP Projects

Designed for University Exams, Product Company Placements & Competitive Coding



TABLE OF CONTENTS — C++ PROGRAMMING

Ch 1. Introduction to C++ & C vs C++ Paradigms	Page 02
Ch 2. Object-Oriented Programming (OOP) Fundamentals	Page 06
Ch 3. Constructors, Destructors & 'this' Pointer	Page 11
Ch 4. Inheritance Modes & Multiple Inheritance Pitfalls	Page 16
Ch 5. Polymorphism, Virtual Functions & VTABLE Mechanism	Page 22
Ch 6. Abstraction, Pure Virtual Functions & Interfaces	Page 28
Ch 7. Templates & Generic Programming (Function & Class)	Page 33
Ch 8. Exception Handling & RAII Pattern	Page 38
Ch 9. Modern C++: Smart Pointers & Lambdas	Page 43
Ch 10. Standard Template Library (STL) Complete Guide	Page 49
Ch 11. Comparison Tables & Mind Maps	Page 56
Ch 12. Real-World Practical Projects (Bank System, Quiz App)	Page 61
Ch 13. Practice Problems (Basic → Advanced)	Page 67
Ch 14. 30+ Important University Viva Questions	Page 71
Ch 15. 30+ Technical Interview Questions (OOP + STL)	Page 76
Ch 16. 1-Day Revision & C++ Cheat Sheet	Page 81

CHAPTER 5: POLYMORPHISM & VTABLE MECHANISM

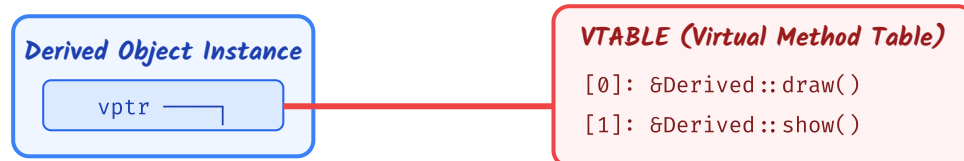


Runtime Polymorphism & Virtual Function Mechanics

1. What is Runtime Polymorphism?

The capability of a base class pointer or reference to invoke overridden methods in derived classes at runtime, determined dynamically based on the actual object being pointed to.

SVG DIAGRAM: VTABLE & VPTR ARCHITECTURE IN MEMORY



2. Complete C++ Code Example

```
#include <iostream>
using namespace std;

class Shape {
public:
    // Virtual method enables dynamic dispatch
    virtual void draw() {
        cout << "Drawing Generic Shape" << endl;
    }
    virtual ~Shape() { cout << "Shape Destructor" << endl; }
};

class Circle : public Shape {
public:
    void draw() override {
        cout << "Drawing Circle ●" << endl;
    }
    ~Circle() { cout << "Circle Destructor" << endl; }
};

int main() {
    Shape* shapePtr = new Circle(); // Base pointer to derived object
    shapePtr->draw();                // Calls Circle::draw() via VTABLE
    delete shapePtr;                 // Calls Circle destructor then Shape
    destructor
    return 0;
}
```

Output: Drawing Circle ● Circle Destructor Shape Destructor

 **Product Company Interview Question:** Why should a base class destructor always be declared virtual when using polymorphism?

Ans: If the base class destructor is not virtual, deleting a derived object via a base class pointer causes **undefined behavior** where only the base destructor is called, leading to memory leaks for derived class members!

CHAPTER 10: STANDARD TEMPLATE LIBRARY (STL)

MASTERY



Complete C++ STL Containers Cheat Sheet

Container	Underlying Data Structure	Access Time	Insertion/Deletion	Use Case
<code>std::vector</code>	Dynamic Contiguous Array	$O(1)$ random access	$O(1)$ amortized end, $O(N)$ middle	Default choice for sequential data.
<code>std::list</code>	Doubly Linked List	$O(N)$ sequential access	$O(1)$ anywhere given iterator	Frequent insertions/deletions in middle.
<code>std::set</code>	Red-Black Tree (Balanced BST)	$O(\log N)$ search	$O(\log N)$ insertion	Unique sorted elements.
<code>std::unordered_map</code>	Hash Table	$O(1)$ average search	$O(1)$ average insertion	Key-value lookup with highest performance.

CHAPTER 12: REAL-WORLD PRACTICAL PROJECT

Project: OOP Bank Account Management System

```

#include <iostream>
#include <vector>
#include <memory>
using namespace std;

class BankAccount {
private:
    int accNum;
    string holderName;
protected:
    double balance;
public:
    BankAccount(int id, string name, double bal) : accNum(id),
holderName(name), balance(bal) {}
    virtual void deposit(double amount) { balance += amount; }
    virtual bool withdraw(double amount) {
        if (amount > balance) return false;
        balance -= amount;
        return true;
    }
    virtual void displayInfo() const {
        cout << "Acc #" << accNum << " | Name: " << holderName << " |
Balance: $" << balance << endl;
    }
    virtual ~BankAccount() = default;
};

class SavingsAccount : public BankAccount {
private:
    double interestRate;
public:
    SavingsAccount(int id, string name, double bal, double rate)
        : BankAccount(id, name, bal), interestRate(rate) {}
    void applyInterest() { balance += balance * (interestRate / 100); }
};

int main() {
    vector<unique_ptr<BankAccount>> bank;
    bank.push_back(make_unique<SavingsAccount>(101, "Alice", 1000.0,
5.0));

    bank[0]→deposit(500);
    bank[0]→displayInfo();
    return 0;
}

```

CHAPTER 14: 30+ UNIVERSITY VIVA & INTERVIEW QUESTIONS

Q1. What is the difference between shallow copy and deep copy?

Ans: Shallow copy duplicates member values as-is (pointer addresses are copied, sharing the same memory block). Deep copy allocates distinct new memory for dynamically allocated pointers and copies the actual values, preventing double-free errors.

Q2. What is RAII in C++?

Ans: Resource Acquisition Is Initialization (RAII) is a C++ idiom where resource ownership (memory, file handles, mutexes) is tied to object lifetime—resources are acquired in constructor and automatically freed in destructor.